

Подробное описание сохранения формы из дизайнера в БД

1. Структура хранения формы в БД

Форма хранится в двух таблицах:

Таблица	Назначение
meta.forms	Основные метаданные формы (название, алиас, связь с сущностью, JSON-состояние)
meta.form_elements	Детальное описание каждого контрола (позиция, размер, свойства, события)

2. Схема таблицы meta.forms

sql

```
CREATE TABLE meta.forms (  
    id SERIAL PRIMARY KEY,  
    cname VARCHAR(255) NOT NULL,          -- Название формы ("Карточка  
персоны")  
    calias VARCHAR(50) NOT NULL UNIQUE,    -- Алиас ("person_edit")  
    identitytypeid INTEGER REFERENCES meta.entitytypes(id), -- Связанная сущность  
    mstate_json TEXT,                     -- JSON-состояние (резервное  
хранилище)  
    created_at TIMESTAMP DEFAULT now(),  
    updated_at TIMESTAMP DEFAULT now()  
);
```

3. Схема таблицы meta.form_elements

sql

```
CREATE TABLE meta.form_elements (  
    id SERIAL PRIMARY KEY,  
    formid INTEGER NOT NULL REFERENCES meta.forms(id) ON DELETE CASCADE,  
    parentid INTEGER REFERENCES meta.form_elements(id), -- для вложенных  
контролов  
    calias VARCHAR(50) NOT NULL,          -- Алиас контрола ("btn_save")  
    cclass VARCHAR(50) NOT NULL,         -- Тип контрола ("textbox",  
"button")  
    cdatasource VARCHAR(255),            -- Источник данных (опционально)  
    mproperties_json TEXT,               -- JSON свойств (позиция, размер,  
binding)  
    lisactive BOOLEAN DEFAULT true,  
    isortorder INTEGER DEFAULT 10  
);
```

4. Формат mproperties_json для контрола

json

```
{
```

```
"x": 10,
"y": 50,
"width": 150,
"height": 30,
"binding_field": "clastname",
"label": "Фамилия",
"is_required": true,
"is_readonly": false,
"is_visible": true,
"events": {
    "on_click": "btn_save_on_click",
    "on_change": "txt_lastname_on_change"
}
}
```

5. Процесс сохранения формы

Шаг 1: Собрать данные из дизайнера

Из `FormDesigner.get_form_data()` получаем:

python

```
{
    'title': 'Карточка персоны',
    'width': 800,
    'height': 600,
    'controls': [
        {
            'id': 'abc123',
            'type': 'textbox',
            'alias': 'txt_lastname',
            'x': 10,
            'y': 50,
            'width': 150,
            'height': 30,
            'properties': {
                'binding_field': 'clastname',
                'label': 'Фамилия'
            }
        },
        {
            'id': 'def456',
            'type': 'button',
            'alias': 'btn_save',
            'x': 10,
            'y': 200,
            'width': 100,
            'height': 30,
            'properties': {}
        }
    ]
}
```

Шаг 2: Сохранить основную запись в `meta.forms`

python

```

def save_form(self, form_data, form_id=None):
    """Сохраняет форму в БД"""

    # Подготовка JSON-состояния (резервное хранилище)
    mstate_json = {
        "class": "MozartForm",
        "objectName": form_data.get('alias', 'Form'),
        "windowTitle": form_data.get('title', 'Новая форма'),
        "geometry": {
            "x": 0, "y": 0,
            "width": form_data.get('width', 800),
            "height": form_data.get('height', 600)
        }
    }

    if form_id:
        # Обновление существующей формы
        sql = """
            UPDATE meta.forms
            SET cname = %s,
                calias = %s,
                mstate_json = %s,
                updated_at = now()
            WHERE id = %s
            RETURNING id
        """
        params = (
            form_data['title'],
            form_data['alias'],
            json.dumps(mstate_json, ensure_ascii=False),
            form_id
        )
        result = self.db.execute_query(sql, params)
    else:
        # Создание новой формы
        sql = """
            INSERT INTO meta.forms (cname, calias, mstate_json)
            VALUES (%s, %s, %s)
            RETURNING id
        """
        params = (
            form_data['title'],
            form_data['alias'],
            json.dumps(mstate_json, ensure_ascii=False)
        )
        result = self.db.execute_query(sql, params)
        form_id = result[0][0] if result else None

    return form_id

```

Шаг 3: Сохранить контролы в meta.form_elements

python

```

def save_controls(self, form_id, controls_data):
    """Сохраняет контролы формы в БД"""

    # Удаляем старые контролы
    self.db.execute_query(
        "DELETE FROM meta.form_elements WHERE formid = %s",
        (form_id,), fetch=False
    )

```

```

)

# Сохраняем новые контролы
for idx, control in enumerate(controls_data):
    # Формируем JSON свойств
    properties = {
        'x': control.get('x', 10),
        'y': control.get('y', 50),
        'width': control.get('width', 150),
        'height': control.get('height', 30),
        'binding_field': control.get('properties', {}).get('binding_field',
    ''),
        'label': control.get('properties', {}).get('label', ''),
        'is_required': control.get('properties', {}).get('is_required',
False),
        'is_readonly': control.get('properties', {}).get('is_readonly',
False)
    }

    # Добавляем события, если есть
    if 'events' in control.get('properties', {}):
        properties['events'] = control['properties']['events']

    sql = """
    INSERT INTO meta.form_elements
    (formid, calias, cclass, cdatasource, mproperties_json, isortorder)
    VALUES (%s, %s, %s, %s, %s, %s)
    """
    params = (
        form_id,
        control.get('alias', f"control_{idx}"),
        control.get('type', 'textbox'),
        control.get('properties', {}).get('datasource', ''),
        json.dumps(properties, ensure_ascii=False),
        idx * 10
    )
    self.db.execute_query(sql, params, fetch=False)

```

Шаг 4: Загрузка формы из БД

python

```

def load_form(self, form_id):
    """Загружает форму из БД"""

    # Загружаем основную запись
    sql = "SELECT id, cname, calias, mstate_json FROM meta.forms WHERE id = %s"
    row = self.db.execute_query(sql, (form_id,))
    if not row:
        return None

    form_data = {
        'id': row[0][0],
        'title': row[0][1],
        'alias': row[0][2]
    }

    # Загружаем контролы
    sql = """
    SELECT id, calias, cclass, mproperties_json, isortorder
    FROM meta.form_elements

```

```

        WHERE formid = %s
        ORDER BY isortorder
    """
    rows = self.db.execute_query(sql, (form_id,))

    controls = []
    for row in rows:
        props = json.loads(row[3]) if row[3] else {}

        controls.append({
            'id': str(row[0]),
            'alias': row[1],
            'type': row[2],
            'x': props.get('x', 10),
            'y': props.get('y', 50),
            'width': props.get('width', 150),
            'height': props.get('height', 30),
            'properties': {
                'binding_field': props.get('binding_field', ''),
                'label': props.get('label', ''),
                'is_required': props.get('is_required', False),
                'is_readonly': props.get('is_readonly', False),
                'events': props.get('events', {})
            }
        })

    return {
        'form': form_data,
        'controls': controls,
        'size': {
            'width': 800,    # можно извлечь из mstate_json
            'height': 600
        }
    }
}

```

6. Интеграция в FormDesigner (методы сохранения/загрузки)

python

widgets/form_designer_old/main_widget.py

```

def save_to_db(self):
    """Сохраняет текущую форму в БД"""
    if not self.db:
        return False

    form_data = self.get_form_data()

    # Подготовка данных
    save_data = {
        'title': form_data.get('title', 'Новая форма'),
        'alias': f"form_{self.form_id}" if self.form_id else "new_form",
        'width': form_data['size']['width'],
        'height': form_data['size']['height'],
        'controls': form_data['controls']
    }

    # Сохраняем форму
    form_id = self._save_form_record(save_data)
    if not form_id:
        return False

```

```

# Сохраняем контролы
self._save_form_controls(form_id, save_data['controls'])

self.form_id = form_id
return True

def load_from_db(self, form_id):
    """Загружает форму из БД"""
    if not self.db or not form_id:
        return False

    data = self._load_form_record(form_id)
    if not data:
        return False

    # Очищаем сцену
    self.clear_form()

    # Устанавливаем размер и заголовок
    self.init_form(
        width=data['size']['width'],
        height=data['size']['height'],
        title=data['form']['title']
    )

    # Загружаем контролы
    for control_data in data['controls']:
        self._create_control_from_data(control_data)

    self.form_id = form_id
    return True

```

7. Куда сохраняются данные при нажатии "Сохранить" в тулбаре

python

```

# В FormDesigner подключаем сигнал
self.toolbar.save_clicked.connect(self.save_to_db)

```

При нажатии кнопки  "Сохранить" на тулбаре:

1. Срабатывает `save_clicked` сигнал
 2. Вызывается `save_to_db()`
 3. Данные сохраняются в:
 - `meta.forms` — основная запись
 - `meta.form_elements` — все контролы
-

8. Пример SQL для проверки сохранённых данных

sql

```
-- Посмотреть все формы
```

```
SELECT id, cname, calias FROM meta.forms;

-- Посмотреть контролы конкретной формы
SELECT fe.id, fe.calias, fe.cclass, fe.mproperties_json
FROM meta.form_elements fe
WHERE fe.formid = 1;

-- Удалить форму (каскадно удалятся и контролы)
DELETE FROM meta.forms WHERE id = 1;
```

Теперь понятно, куда и как сохранять?